NOVACART

E-Commerce Store Mini Project Report

Student	Bandara K.M.S.N
Registration Number	EG/2023/5515
Course	EC4307 Web Application Development
Project Topic	E-Commerce Store

Project summary

NovaCart is a responsive e-commerce store prototype built with Python Flask, SQLite, HTML/Jinja templates and CSS. Customers can browse products, search/filter/sort the catalogue, add items to a cart and place a demo order. Store operators can create, read, update and delete products and review order records.

Important: complete the testing table and add genuine screenshots from your own running application before submission. Do not report tests as passed until you have performed them.

1. Introduction and objectives

The project demonstrates core e-commerce workflows and database-backed product management. Objectives are to provide a simple shopping interface, implement full product CRUD, validate user input, persist records in a relational database and demonstrate basic order processing.

2. Technologies

• Python and Flask — server-side routes, request handling and validation.

• SQLite — persistent relational storage.

• HTML/Jinja — dynamic pages and forms.

• CSS — responsive presentation.

• Browser session — temporary cart state.

3. Functional requirements

• Browse product listings and product details.

• Search by name, description or category; filter by category; sort by name or price.

• Create, view, update and delete product records.

• Add items to cart and update quantities.

• Collect demo customer details and place a simulated order.

• Save order and order items, reduce inventory, and view order details.

4. Database design

The database contains three tables. products stores product information and stock. orders stores customer information and order totals. order_items stores the purchased product name, quantity and unit price. Primary keys identify records; foreign keys link order items to orders and products.

PRODUCTS	ORDER_ITEMS	ORDERS
id (PK)	id (PK)	id (PK)
name, category	order_id (FK)	customer_name, email
description, price	product_id (FK, nullable)	address, total
stock, image_url	product_name, quantity	created_at
	unit_price	

5. Implementation and validation

Flask routes provide the storefront, product details, cart, checkout, product forms and order views. SQL uses parameterized values. Server-side checks validate required fields, non-negative prices and stock, quantity availability and a basic email format. Checkout rechecks stock before writing order data and updating inventory.

6. UI/UX and responsiveness

The interface uses consistent navigation, product cards, clear action buttons, feedback messages and responsive CSS media queries. Optional image URLs are supported; a designed placeholder appears when no image is supplied.

7. Testing plan

Execute these tests locally and fill in the actual result column. Capture genuine screenshots from the running application for the report.

Test	Expected result	Actual result
Create valid product	Saved and visible	To be completed
Invalid price/stock	Validation error	To be completed
Search/filter/sort	Correct matching results	To be completed
Edit product	Changes persist	To be completed
Delete product	Product removed	To be completed
Checkout within stock	Order saved; stock reduced	To be completed
Quantity over stock	Request rejected	To be completed
Mobile viewport	Layout adapts	To be completed

8. Screenshots to add

• Storefront with product cards

• Search/filter results and product detail

• Add/edit product form and product management list

• Cart and checkout confirmation

• Order list and order details

9. Limitations and future improvements

This is a learning prototype. It does not include administrator authentication, real payment processing, email notifications, CSRF protection or production deployment hardening. Future work can add role-based access, secure deployment, automated tests, payment integration, email notifications and order status management.

10. Conclusion

NovaCart demonstrates the core product catalogue and demo order workflow with persistent database storage and full product CRUD. Before submission, run the app, complete the test table with actual results and insert genuine screenshots.

Run instructions

Install Python 3.10+, run `pip install -r requirements.txt`, then `python app.py`. Open `http://127.0.0.1:5000`. See README.md for details.